Memoria 2: Pipeline y empaquetado

Autor: Jeferson Torrado Bayona

Fecha: Noviembre de 2026

Repositorio: https://github.com/JefersonTB/proyecto-mlops-2026-II

1 Objetivo

Construir un prototipo MLOps para anticipar caídas de producción de petróleo a 30 días, integrando entrenamiento e inferencia con Prefect, seguimiento y registro con MLflow y servicio de predicción con FastAPI. Se reutilizó el entorno Poetry de la actividad 1.

2 Descripción del trabajo

Se utilizó el archivo Volve production data.xlsx, consultado mediante el recurso de Kaggle https://www.kaggle.com/code/imranulhaquenoor/volve-field-production-data-analysis-eda/input . La hoja Daily Production Data contiene 15.634 registros y 24 columnas. Se seleccionaron registros WELL_TYPE=OP; se excluyeron los de inyección WI. Se comprobó que no hubiera duplicados de pozo y fecha. Los días ausentes conservaron valores faltantes y los ceros de producción se conservaron.

La etiqueta identifica si la media de petróleo de los 30 días siguientes es inferior al 80 % de la media de los últimos 30 días. Se requieren historial y futuro completos y una base de producción positiva. El horizonte es móvil de 30 días, no un mes calendario. Las variables explicativas utilizan únicamente información disponible al cierre de la fecha de predicción.

El módulo src/datos_produccion.py prepara variables y separa fechas. El entrenamiento flows/entrenamiento_logistica.py utiliza las tareas de ingesta, evaluación y registro de flows/entrenamiento_produccion.py. La inferencia está en flows/inferencia_produccion.py; los deployments en flows/desplegar.py; la API en api/main.py. Los parámetros se separan en YAML dentro de configs/.

3 Decisiones técnicas

Se utilizó separación temporal para simular predicciones futuras: entrenamiento hasta el 26/05/2014, validación desde el 26/06/2014 hasta el 22/06/2015 y prueba desde el 23/07/2015 hasta el 18/08/2016. Se dejaron 30 días de separación para evitar que las etiquetas de un periodo alcanzaran el siguiente. Se descartó invertir el orden temporal y se mantuvo el horizonte de 30 días frente a las alternativas diaria o anual.

Después de explorar la validación se restringió el alcance a pozos con al menos 365 ejemplos, 30 caídas y 30 casos sin caída en entrenamiento: F-12 y F-14. Esta decisión se tomó durante el desarrollo y limita la independencia de la selección. El modelo final emplea 2.234 ejemplos de entrenamiento y 721 de validación, regresión logística con ponderación de clases y 11 entradas: diez variables numéricas y la identidad del pozo. Las variables numéricas se imputan y escalan; la identidad se codifica como categoría.

MLflow conserva el candidato de cada entrenamiento. Solo crea una versión y actualiza el alias champion cuando el AUC de validación supera 0,70 y al champion vigente. Se probaron selecciones ALL/NUMERIC/CUSTOM; el modelo final utiliza CUSTOM. La inferencia resuelve el alias en cada ejecución y fija la versión durante el lote. La API lo resuelve al iniciar y requiere reinicio si cambia.

4 Evidencias

Los archivos están en docs/evidencias/actividad-2/: api_entrada.json, api_respuesta.json, api_rechazo.json, evaluacion_prueba_v1.json, diagnostico_por_pozo_v1.csv, comparacion_temporal.csv y comparacion_exploratoria_periodo_posterior.csv, además de la captura disponible de Prefect (Ver **Figura 1**).

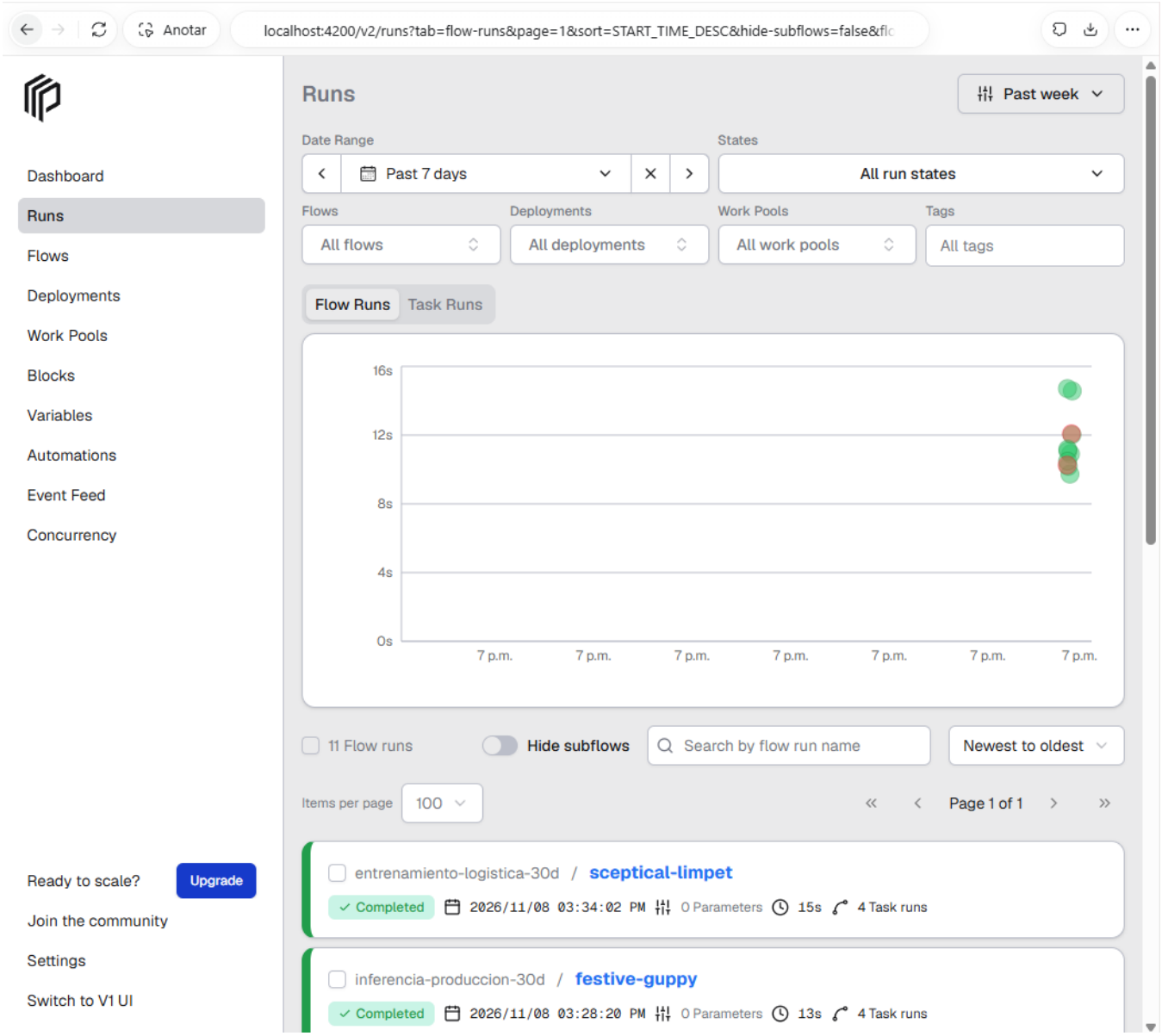


Figura 1. Screencapture en Prefect de la ventana Runs

En Prefect se observaron inferencia festive-guppy completada y entrenamiento sceptical-limpet con tareas completadas y decisión de conservar el champion al repetir el mismo AUC. Los deployments son entrenamiento-manual e inferencia-diaria; la inferencia está programada a las 08:00, zona America/Bogota. La ejecución diaria requiere equipo, servidores y proceso de servicio activos. El lote reproduce la fecha histórica 22/06/2015; no representa datos de producción en vivo.

La API respondió HTTP 200 para F-12, score 0,3075698113, clasificación 0 y versión 1. El archivo de rechazo documenta la prueba de un pozo fuera del alcance.

5 Resultados

Métrica	Champion versión 1
AUC validación	0,7015
Average precision validación	0,2249
AUC prueba (779 ejemplos)	0,4996
Average precision prueba	0,3528

Precisión con umbral 0,5	0,2508
Recall con umbral 0,5	0,4634

La prueba contiene 164 caídas: 76 detectadas, 88 omitidas y 227 falsas alarmas. El AUC por pozo fue 0,6177 para F-12 y 0,3781 para F-14. No se demostró generalización suficiente para uso operativo; el score tampoco se considera una probabilidad calibrada.

Como exploración adicional se compararon regresiones y bosque en dos ventanas temporales válidas; una tercera ventana no cumplió los mínimos. Un candidato de regresión individual por pozo, C=0,01, obtuvo AUC de validación 0,8270, pero solo 0,5344 en el periodo posterior ya observado. Su average precision posterior fue 0,3550. Se guardó el paquete como artefacto, verificando scores idénticos después de cargarlo, y no se desplegó. Estas comparaciones posteriores a inspeccionar la prueba son exploratorias.

6 Problemas encontrados

Se corrigió la ruta del Excel. El guardado inicial del modelo requirió declarar tipos específicos de skops para el modelo entrenado localmente. Una ejecución desplegada falló al imprimir un emoji de MLflow con codificación Windows cp1252; se establecieron PYTHONUTF8=1 y PYTHONIOENCODING=utf-8 y la nueva ejecución completó. Se distinguieron ventanas de servidores y terminal de trabajo para evitar bloquear comandos.

El principal resultado inesperado fue la caída del AUC en fechas posteriores. Se diagnosticó por pozo y se probaron alternativas, sin atribuir una causa no comprobada. No se realizó una comprobación completa desde un clon limpio. Tampoco se demostró el criterio adicional de reemplazar un champion por una segunda versión mejorada y reiniciar la API con esa nueva versión; sí se verificó la conservación del alias ante empate.

7 Conclusiones

Se integraron pipelines, registro, inferencia programada y API. La ejecución correcta de infraestructura no garantiza utilidad predictiva. El champion se conserva como prototipo académico; se requiere revisar cambios operativos, calidad de datos y estabilidad temporal, además de reservar datos nuevos para una confirmación independiente. No se continuó experimentando para priorizar una entrega reproducible y transparente.

8 Uso de herramientas de IA

Se utilizó ChatGPT/Codex para orientar el diseño, generar bloques de Python y PowerShell, interpretar errores y métricas. La contribución individual incluyó elegir el dominio de producción de petróleo y el horizonte, ejecutar comandos, operar los servicios, aportar capturas y revisar los resultados junto con la asistencia. No se presenta el código ni esta memoria como trabajo realizado sin IA. Se corrigieron rutas, serialización y codificación durante las pruebas.